HACKTHEBOX

INTRODUCTION TO DYNAMIC ANALYSIS WITH WINDBG

CHEAT SHEET

HTB ACADEMY CHEATSHEET

Debugger Commands

Command	Description
`.hh`	Opens the debugger help documentation.
`.readmem`	Reads memory from a specified address and saves it to a file.
`.writemem`	Writes data from a file to a specified memory address.
`.reload`	Reloads the symbol files for the current target.
`.reload [/f]`	Forces a reload of symbol files, even if they are already loaded.
`.symfix <path>`	Automatically sets the symbol path to the Microsoft symbol server.
`.sympath`	Displays or sets the symbol search path.
`.formats <addr>`	Displays a memory address in multiple formats (hex, decimal, binary, etc.).
`.attach 0n<PID>`	Attaches the debugger to a running process by its Process ID (PID).
`.detach`	Detaches the debugger from the current process.
`.restart`	Restarts the current target process.

Command	Description
`.reboot`	Reboots the target machine in kernel debugging mode.
`.printf`	Prints formatted output to the debugger console.
`.echo`	Displays a message in the debugger console.
`.shell`	Executes a shell command from within the debugger.
`q`	Quits the debugger.
`.cls`	Clears the debugger console screen.
`;`	Executes multiple commands in sequence (e.g., `command1;command2`).
`n <base>`	Sets the default number base for output (e.g., `n 16` for hexadecimal).
`!analyze [-v]`	Analyzes the current exception or bug check; `-v` for verbose output.
`!vad <VadRoot>`	Displays the Virtual Address Descriptor (VAD) tree for a given root address.
`.lastevent`	Displays information about the last event that occurred in the debugger.

Functions & Symbols

Command	Description
`u <addr>`	Disassembles instructions starting at the specified address.
`uf <addr>`	Disassembles a function starting at the specified address.
`ln <addr>`	Lists the nearest symbols for a given address.
`x <FunctionName>`	Examines symbols matching a specific pattern (e.g., `x *!*`).

HTB ACADEMY CHEATSHEET

Execution Flow Control

Command	Description
`g`	Continues execution of the target process.
`g <addr>`	Continues execution until the specified address is reached.
`gu`	Executes until the current function returns.
`p`	Steps over the next instruction (executes a single instruction).
`t`	Steps into the next instruction (traces into calls).

Threads & Call Stacks

Command	Description
`~`	Lists all threads in the current process.
`~ [thread_id]`	Switches to or displays information about the specified thread.
`k [NumOfFrames]`	Displays the call stack for the current thread; optional number of frames.
`kv`	Displays the call stack with additional frame information (e.g., parameters).
`~*k`	Displays the call stack for all threads in the current process.
`r`	Displays the current state of all registers.
`r [register(s)]`	Displays the value of the specified register(s).
`r [register]= [value]`	Sets the value of the specified register.
`rX`	Displays or modifies extended registers (e.g., `rXmm0` for XMM registers).

HTB ACADEMY CHEATSHEET

Data, Expression & Types

Command	Description
`dt [-rN] [type] <addr>`	Displays type information for a symbol or structure at the specified address; `-rN` for recursion depth.
`??`	Evaluates and displays a C++ expression.
`? [-rN] <expr>`	Evaluates and displays a MASM expression; `-rN` for recursion depth.
`poi(<addr>)`	Dereferences and displays the pointer at the specified address.

Breakpoints

Command	Description
`bp <addr>`	Sets a breakpoint at the specified address.
`bl`	Lists all active breakpoints.
`bc <id>`	Clears the breakpoint with the specified ID.
`bp <addr>;.printf ...;g`	Sets a breakpoint, executes a `.printf` command when hit, and continues execution.

Modules & PE Headers

Command	Description
`lm`	Lists loaded modules (executables and DLLs) in the target process.
`lm f`	Lists loaded modules with detailed information (flags, timestamps, etc.).
`lm fm <ModuleName>`	Lists information about a specific module by name.
`!dh`	Displays the header information of a loaded module.

HTB ACADEMY CHEATSHEET

Command	Description
`!dh -i <BaseAddr>`	Displays import table information for a module at the specified base address.
`!dh -e <BaseAddr>`	Displays export table information for a module at the specified base address.

Read Memory

Command	Description
`db <addr>`	Displays memory as bytes (8-bit values) starting at the specified address.
`dw <addr>`	Displays memory as words (16-bit values) starting at the specified address.
`dd <addr>`	Displays memory as double-words (32-bit values) starting at the specified address.
`dq <addr>`	Displays memory as quad-words (64-bit values) starting at the specified address.
`dp <addr>`	Displays memory as pointer-sized values starting at the specified address.
`dS <addr>`	Displays memory as a null-terminated ANSI string starting at the specified address.
`du <addr>`	Displays memory as a null-terminated Unicode string starting at the specified address.
`dqs <addr>`	Displays memory as quad-words (64-bit values) and interprets them as symbols.
`dps <addr>`	Displays memory as pointer-sized values and interprets them as symbols.
`dyb <addr>`	Displays the contents of a memory block at the specified address in binary format.

HTB ACADEMY CHEATSHEET

Write Memory

Command	Description
`eb`	Writes a byte (8-bit value) to the specified memory address.
`ew`	Writes a word (16-bit value) to the specified memory address.
`ed`	Writes a double-word (32-bit value) to the specified memory address.
`eq`	Writes a quad-word (64-bit value) to the specified memory address.
`ep`	Writes a pointer-sized value to the specified memory address.

Time Travel Debugging

Command	Description
`p-`	Steps backward over the previous instruction (undoes a `p` command).
`t-`	Steps backward into the previous instruction (undoes a `t` command).
`g-`	Continues execution backward (time travel debugging).
`g- <addr>`	Continues execution backward until the specified address is reached (time travel debugging).
`!tt AA:BB`	Analyzes time travel trace data between two points (AA and BB).
`@$cursession.TTD`	Provides access to Time Travel Debugging (TTD) objects data for the current session.
`@$cursession.TTD.Calls`	Displays function calls recorded in the TTD trace.

Command	Description
`@$cursession.TTD.Memory`	Provides access to memory snapshots recorded in the TTD trace.
`@$cursession.TTD.Data.Heap`	Provides access to heap memory data recorded in the TTD trace.
`@$cursession.TTD.Data.Utility.GetHeapAddress`	Retrieves the heap address for a specific object in the TTD trace.
`@$curprocess.TTD`	Provides access to Time Travel Debugging (TTD) data for the current process.
`@$curprocess.TTD.Threads`	Lists threads recorded in the TTD trace for the current process.
`@$curprocess.TTD.Events`	Displays events recorded in the TTD trace for the current process.
`@$curthread.TTD`	Provides access to Time Travel Debugging (TTD) data for the current thread.
`@$curthread.TTD.Info`	Provides detailed information about the current thread in the TTD trace.

Debugger Objects & Built-in Variables

Command	Description
`dx [-rN] [-g] <expr>`	Evaluates and displays an extended expression using expression model; `-r` for recursion depth, `-g` for gridded output.
`dx @$var [= value]`	Evaluates or assigns a value to a debugger variable.
`@$curthread`	Represents the current thread in the debugger.

HTB ACADEMY CHEATSHEET

Command	Description
`@$curprocess`	Represents the current process in the debugger.
`@$curstack`	Represents the current stack in the debugger.
`@$curframe`	Represents the current stack frame in the debugger.
`@$cursession`	Represents the current debugging session.
`@$curregisters`	Represents the current register values in the debugger.
`@$scripts`	Lists the currently loaded scripts in the debugger.
`@$scriptContents`	Displays the contents of a loaded script.
`@$scriptContents.FunctionName()`	Calls a specific function defined in the loaded script.
`@$proc`	Represents the current process context in the debugger.
`Debugger.State`	Provides access to the current state of the debugger.
`Debugger.State.DebuggerVariables`	Lists all debugger variables and their values.
`Debugger.Sessions`	Provides access to all active debugger sessions.
`Debugger.Utility`	Provides utility functions for debugging tasks.
`$peb`	Represents the Process Environment Block (PEB) of the current process.
`$teb`	Represents the Thread Environment Block (TEB) of the current thread.
`$exentry`	Represents the entry point of the current executable.

HTB ACADEMY CHEATSHEET

Command	Description
`$tpid`	Represents the Thread Process ID (TPID) of the current thread.
`$tid`	Represents the Thread ID (TID) of the current thread.

Utility Functions (`Debugger.Utility`)

Function	Description
`Collections.FromListEntry`	Creates a collection from a linked list structure in memory.
`Collections.CreateArray`	Creates an array from a specified memory range or data.
`Code.CreateDisassembler`	Creates a disassembler object to analyze instructions at a given address.
`Control.ExecuteCommand`	Executes a debugger command from within a script or expression.
`Control.SetBreakpointForReadWrite`	Sets a breakpoint triggered on read/write access to a memory location.

Language Integrated Query (LINQ)

LINQ Method	Description
`Select`	Projects each element of a collection into a new form.
`Where`	Filters elements based on a specified condition.
`All`	Checks if all elements in a collection satisfy a condition.
`Any`	Checks if any element in a collection satisfies a condition.
`ToDisplayString("sb")`	Converts the result to a formatted string with specified options.

HTB ACADEMY CHEATSHEET

LINQ Method	Description
`SelectMany`	Projects each element to a collection and flattens the results.
`First`	Returns the first element of a collection.
`Last`	Returns the last element of a collection.
`Single`	Returns the only element of a collection, or throws an exception if there are multiple.
`Min`	Returns the minimum value in a collection.
`Max`	Returns the maximum value in a collection.
`Count`	Returns the number of elements in a collection.

Scripts Commands

Command	Description
`.scriptload <path>`	Loads a script from the specified file path into the debugger.
`.scriptrun <name>`	Executes a loaded script by its name.
`.scriptunload <name>`	Unloads a script from the debugger by its name.

HTB ACADEMY CHEATSHEET